THE QUANT CLUB · IIT (BHU) VARANASI

HIDDEN HAND

Roadmap v2

Wednesday and Thursday. Learn rival_1 first (the clearer bot), then rival_2 as its upgrade, decide the entry on Thursday at noon with simulation and live arena data, improve the chosen bot in the afternoon, and submit Thursday evening.

Replaces	Days 2-3 of the Solo Roadmap (Tuesday's steps S0-S5 are done)
Study	zoo/rival_1.py (751 lines) on Wednesday, zoo/rival_2.py (686 lines) on Thursday morning
Entry	moneyheist_bot.py: rival_2 for now; final base chosen Thursday 12:00
Uploads close	Thursday 8 October 2026, end of day (exact time on the WhatsApp community)

1. The verdict: why the plan changes

Measured with tools/eval.py, 500 deals, seeds 1 and 2, every bot against the same pool (house bots and zoo, itself excluded). Max call measured with tools/speed.py in one quiet process.

Bot	Pool	Wtd.	vs sharp	vs rival_1	vs rival_2	vs rival_3	vs old_plan	Max call
sharp (our old entry)	-4.7	-7.3	0.0	-20.9	-21.2	-19.9	-8.0	0.1 ms
old_plan (old Wed plan)	+5.2	+4.3	+8.0	+2.6	-3.7	+0.9	-	0.1 ms
rival_1	+13.11	+11.68	+20.6	-	+2.8	+3.9	-2.6	0.1 ms
rival_2	+13.27	+11.61	+20.7	-2.8	-	-1.1	+3.6	1.4 ms
rival_3	+13.42	+11.95	+19.7	-3.9	+1.1	-	-0.7	0.6 ms

- The rivals beat sharp by about **+20 score (+230 PnL per deal)**. The old Wednesday plan (S6-S10) was built and measured as zoo/old_plan.py: it reaches +8 against sharp. Small fixes cannot close a gap of 20.
- The three rivals tie on the pool (13.1 to 13.4), so the choice depends on what the real field looks like.

Why rival_2, not rival_1

Opponent type	rival_1	rival_2	Better
ai_typical (a one-shot AI bot)	+8.4	**+15.6**	rival_2
old_plan (a careful team, noisy quotes)	-2.6	**+3.6**	rival_2
reader	+19.3	**+22.6**	rival_2
hider	**+11.7**	+5.3 (**+11.1 after B1**)	tie after B1
rival_1 / rival_3 head to head	-	-2.8 / -1.1	rival_1 (target of B0)

Re-weighting the same numbers for three possible fields: mostly average teams (rival_1 +8.6, rival_2 **+11.3**); full of strong AI bots (rival_1 **+7.4**, rival_2 +5.5); many hiders (rival_1 +8.9, rival_2 +8.4). This is a freshers' event: most bots will look like ai_typical or old_plan, so rival_2 fits the likely field best. Its losses to rival_1 and rival_3 are the main thing Part B fixes.

Is this allowed?

Yes. rival_2 was written by an AI agent for our team, and AI tools are allowed. It is not another team's code. The one condition: any member must be able to explain any part of the file. That is what Part A is for. It goes on the live board Wednesday night as a placeholder; by the final upload you must be able to explain every line of whichever base you submit.

What we already know about rival_2 (measured Wednesday morning)

Finding	Evidence	Step
Fooled by hiders. A hider quotes the public estimate, which looks exactly like an honest bot holding an average hand, so rival_2 keeps believing 'average hand'.	Hand estimate 7-10 points off against hider (rival_1: 2.5). Ignoring quotes equal to the public estimate: **+5.7 vs hider, +0.38 pool**, nothing else hurt.	B1
Loses as the maker against rival_1 and rival_3. Its quotes get picked off; it wins as the taker.	Against rival_1, per game: maker -42.5 (round 1 alone -15.4), taker +21.8.	B0
Not the cause: its paid manipulation, quote noise, or the first quote revealing its hand.	Turning off probes: +0.14 vs rival_1 but -0.39 pool. Noise ±0.5: -0.27. Hand-hiding first quote: 0.0 pool (+0.6 vs rival_1, -1.0 vs rival_3).	do not retry

The plan: learn both, decide with data

- **rival_1 first.** Its ideas are the clearest: exact identification, steer and harvest, a 1-D filter. rival_2 is the same ideas generalised (trust as an unknown, a 2-D filter, simulated play-outs), so after rival_1 most of rival_2 reads as 'the same thing, more general'.

- **rival_2 is the entry until Thursday noon,** because it fits the likely field best. It goes on the live board Wednesday night so the arena tells us how it does against real teams.
- **Thursday 12:00, gate G:** pick the final base from simulation numbers, live arena results, and which bot you can explain better. Then spend the afternoon improving only that one.

What we found on Wednesday morning: rival_1's blind spot

zoo/old_plan.py adds ±0.5 random noise to its quotes. With that noise, it beats rival_1 by +2.6; without it, it loses by -2.0. Diagnosis (300 deals against old_plan):

- **Round 1:** in 14% of decisions rival_1 locks onto a WRONG exact hand: the noise happens to pass its 0.01 whole-number test, and it then plays as if it knew the hand.
- **Rounds 2-5:** it gives up on exact mode and uses the fallback filter. Its estimate of their hand is about 3 points off in rounds 1-2 but **7-9 points off from round 3**, no better than a blind guess. Against hider the same fallback gets to about 2.5. Something in the later quotes misleads it.
- **The obvious fix fails:** waiting for two quotes before trusting exact mode costs -1.4 pool score (it throws away round-1 profit against every reader) and does not help against old_plan. Do not retry it.

2. How rival_1 works: the five ideas in plain words

Idea	What it means	Where in rival_1
1. Exact identification	A reader's quote is a + b·S, a straight line in its hand sum S. Quotes are rounded to 0.01 and S is a whole number, so inverting one quote gives S almost exactly. Trying a grid of trust values and keeping only those that stay consistent across quotes leaves one answer, usually after 1-2 quotes.	coef(), _filter_quote()
2. Steering	Quote just inside their no-trade band (their fair ± 1.97). They do not trade, but they read the quote as our hand and move their belief, so their next quote lands on the wrong side of the truth.	_plan(), BAND
3. Harvesting	Quote just outside their band, so they trade 2 lots at about their fair value, which we know is wrong.	_plan(), _hk()
4. A planner	For the remaining quotes, try every mix of steer and harvest in both directions, fill the 10-lot limit with the best trades, and play the first step of the best plan.	_plan()
5. A fallback filter	If no exact hypothesis survives (hiders, liars, other rivals), keep a Gaussian belief over S, updated by their quotes and by their buys, sells and passes; quote around THEIR fair so our quote reveals little about our hand.	_gauss(), _state(), _quote()

rival_2 and rival_3 use the same core ideas with different machinery. rival_2 adds one trick rival_1 lacks (an extreme quote when at the position limit). rival_3 adds exact models for hider, liar and ai-style bots, and predicts the opponent's swap. These become improvement experiments in Part B.

What rival_2 adds on top of rival_1

Idea	What it means	Where
1. The opponent's fair value is a formula	Most bots value the stock as their hand O + board + (their guess of our hand) + hidden cards × average. Their guess of our hand moves a fraction t (trust) toward what our last quote implies. Written out, their fair = A0 + A1·t + (B0 + B1·t)·O: known numbers, plus two unknowns, O and t.	coefs()
2. Many hypotheses about HOW they think	Does it read our quotes at all? With which rule (raw, clamped to a legal hand, ignoring silly quotes, or also reading the price it answers)? How honest are its own quotes (lam: 1 honest, 0 meaningless)? Does it react to our trades (tau) or expect our swap (kappa)? Every combination is one hypothesis.	_init()
3. A Kalman filter per hypothesis	Each hypothesis keeps a 2-D normal belief over (O, t). An opponent quote is a noisy reading of their fair; their buy / sell / pass on our quote says their fair is above / below / within 2 of our price. Each event updates the belief and re-weights the hypothesis by how well it predicted the event.	ekf(), _see()
4. Decisions by play-out	For each candidate price (or buy / pass / sell), simulate the rest of the deal under the top 6 hypotheses and score the result with a concave utility (like the square-root score: big swings count less).	_roll(), _quote(), _respond()
5. Steering readers	At +10 or -10 lots, a quote of 195 or 0 cannot be traded against in the bad direction, but a reader believes it and moves its quotes. Paid 'probes' only when sure the opponent is a reader.	_quote()

3. Schedule

Times are a guide; the deadline is end of day Thursday (check the exact time on the WhatsApp community). Work back from it: freeze 3 hours before, upload 1 hour before.

When	Step	Who	Time
Wed, now	W1 Entry = rival_2, trace tool for both bots	Claude + you	45 min
Wed	M1-M6 Learn rival_1 (zoo/rival_1.py)	You learn	about 5 h
Wed night	N1 Upload rival_2 (+ B1 if done), start red-team agents in the background	You + Claude	30 min
Thu morning	Scouting: what did the live board say overnight?	You	20 min
Thu morning	R1-R6 Learn rival_2 as an upgrade of rival_1	You learn	about 4 h
Thu 12:00	G Gate: choose the final base	You decide	15 min
Thu afternoon	W3 Improve the chosen base (experiment loop)	You code	3 h
Thu afternoon	W5 Tune, confirm, upload	Claude runs, you decide	45 min
Deadline - 3 h	T1-T2 Freeze, scouting, harden	You	75 min
Deadline - 2 h	T3 Final walkthrough: you explain, Claude corrects	You	60 min
Deadline - 1 h	T4 Final upload	You	15 min

If you fall behind

- Learning: M2, M4 (rival_1) and R2, R4 (rival_2) may be done at the 'what each function does' level. For the bot you finally submit, finish them line by line in T3.
- Improving: cut from the bottom of the B list. Keep B1 for whichever base you choose (each is already measured).
- Never cut: N1 (a safe upload Wednesday night), G, T2, T4.

Targets for the chosen base

Measure (tools/eval.py, 500 deals, seeds 1 2)	rival_2 base	rival_1 base
Baseline pool / weighted (copy, with itself in the pool)	+12.39 / +10.39	+12.23 / +10.46
Quick measured gain (B1)	hider detector: +0.38 pool	extreme quote at limit: +0.26 pool
Main weakness to fix (B0)	vs rival_1 -2.83, vs rival_3 -1.14 (maker losses)	vs old_plan -2.57 (noisy quoters)
Target	both head to heads above 0	old_plan above 0
Never drop	old_plan, ai_typical, reader	rival_2, rival_3 head to heads
Slowest call (tools/speed.py)	1.4 ms → keep under 10 ms	0.1 ms → keep under 10 ms

4. Wednesday: set up, then learn rival_1

W1. Entry = rival_2, trace tool for both bots

TOGETHER · **Time:** 45 min · **Goal:** a safe entry and a microscope for either bot's brain

1. Your moneyheist_bot.py is currently a copy of rival_1. Make the entry rival_2: `cp zoo/rival_2.py moneyheist_bot.py`.
2. Replace the three header lines with your real team name, members and mentor.
3. Study rival_1 from zoo/rival_1.py itself (unchanged, so the line numbers in M1-M6 match).
4. Validate and run the baseline below. Expect +12.39 pool, 0 against rival_2, -2.83 against rival_1.
5. Commit as "W1: entry from rival_2".

RUN IT YOURSELF (terminal)

```
python backtester.py --validate moneyheist_bot.py
python tools/eval.py moneyheist_bot.py --deals 500 --seeds 1 2
```

PROMPT (paste into Claude Code)

```
Create tools/trace.py (local only). Usage:
  python tools/trace.py BOT OPP [--seed N] [--deal D] [--swap]
It plays ONE deal (engine.play_deal, seeded like play_match deal D, --swap = seats swapped)
with BOT wrapped so that after every move/quote/respond call it prints: round, call, what BOT
returned, the opponent's TRUE hand sum and the TRUE final value, and BOT's internal state read
from whichever attributes exist:
- rival_1 style (self.hyps): number of hypotheses, self.dead, self._best() (trust, S), the E
  from self._state(obs), and for quotes whether it was a steer or a harvest (vs _their_F);
- rival_2 style (self.H): number of hypotheses, the top 3 by weight (reads, rule, lam, tau,
  kap, H_MO, H_MT, share of weight), self._mo(), self._p_reader(), E = alpha + beta * _mo(),
  and for quotes whether it was an extreme (195 / 0), a probe, or a normal quote.
Print the describe() replay at the end. Read-only: never change the bot. Keep it simple and
commented so I can read it. Then run it for zoo/rival_1.py and for moneyheist_bot.py against
reader, seed 3, deal 0.
```

Your part

- Run WALKTHROUGH on tools/trace.py (it is short).
- In the rival_1 trace, find the call where the hypotheses collapse to one, and the first steer quote.

How every module works

1. **Read** the lines yourself first (10 minutes, no AI). Write down what you think each function does.
2. **WALKTHROUGH** those lines with Claude, 20 lines at a time. You summarise each chunk.
3. **Do the exercise.** It makes you use the code, not just read it.
4. **Answer the UNDERSTAND IT box** out loud, then 3 lines in NOTES.md in your own words.

WALKTHROUGH (module version)

```
Walk me through <zoo/rival_1.py or zoo/rival_2.py> lines <A>-<B> line by line, about 20 lines
at a time. For each line: what it does, and the value it produces in this concrete situation:
<seed 3, deal 0, against reader, round <r>> (use tools/trace.py to get real numbers).
After each chunk, stop and ask me to summarise it in one sentence. If my summary is wrong
or vague, re-explain the unclear line. Continue only when I say "next".
```

Optional warm-up (30 min): zoo/old_plan.py's view_of_me() and BOUNDS block are the simple versions of rival_1's coef() trust term (M1) and the buy/sell/pass update in _gauss (M4).

UNDERSTAND IT · answer out loud before the next step

1. Why do we learn rival_1 first, even though rival_2 is the likely entry?
2. Which file is the entry right now, and which files do you study?
3. Why is copying a rival allowed, and what does the rules' 'explain any line' clause demand of you?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for W1 in NOTES.md in your own words.

M1. The core maths: coef() (lines 1-93)

YOU LEARN · **Time:** 45 min · **Goal:** know why one quote reveals a whole hand

- Lines 1-42: the docstring. It is the bot's whole strategy in 40 lines; read it twice.
- Lines 46-66: the constants. For each, say what it controls (they become tuning knobs in W5).
- Lines 70-76: Phi and phi, the normal distribution's CDF and density, used by the fallback.
- Lines 78-92: coef(), the most important function in the file.

Exercise (pen and paper)

1. Take seed 3, round 1: reader holds 3 5 6 Q 4 (S = 30) and reader's quote after seeing a quote of 102.64 was 97.9 (from Tuesday's S1).
2. Rewrite reader.py's fair value with S as an unknown. Show it is a + b·S, and find a and b.
3. Now solve for S from the quote 97.9. You should land on 30 (up to rounding).
4. Compare your a and b with what coef(0.8, 0, 0, 0, 0, 102.64, 0, 0) returns (ask Claude to run it).

UNDERSTAND IT · answer out loud before the next step

1. Why is a reader's quote a straight line a + b·S in its hand sum S?
2. Quotes are rounded to 0.01 and S is a whole number. Why does that let rival_1 find S exactly?
3. In coef(), b = 1 - K/N. Why is b below 1 for a naive opponent (t = 0) but above 1 for a reader with t = 0.8?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for M1 in NOTES.md in your own words.

M2. Bookkeeping: replaying the history (lines 95-213)

YOU LEARN · **Time:** 60 min · **Goal:** know how the bot turns obs into an ordered list of events

- __init__ (96-111): every field the bot remembers within one deal.
- _rank, _held (116-125): which card was in which slot, before and after our swap.
- _view (127-142): what the opponent could see at the moment of a past quote.
- _sync (144-167): processes only new tape entries, and each round's moves before that round's quotes.
- _move_ev, _branch_peek, _tape_ev (169-212): what happens on an opponent peek, swap, quote or answer.

Exercise

RUN IT YOURSELF (terminal)

```
python tools/trace.py zoo/rival_1.py zoo/swapper.py --seed 1 --deal 0
```

Find the opponent's swap in the output. Explain what _move_ev did to every hypothesis at that moment (the pend field) and why the next quote is still consistent.

UNDERSTAND IT · answer out loud before the next step

1. Why does _sync process each round's peeks and swaps before that round's quotes?
2. What does _view return, and why must it describe the moment of the quote rather than now?
3. What is a hypothesis tuple (t, pk, S, pend)? Explain each field.

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for M2 in NOTES.md in your own words.

M3. Exact identification (lines 214-269)

YOU LEARN · **Time:** 45 min · **Goal:** know when the bot knows their hand exactly, and when it gives up

- _pk (214-221): the sum and count of our cards the opponent peeked.
- _filter_quote (223-246): the heart of identification. Keep a hypothesis only if the implied S is a whole number within tolerance and consistent with earlier quotes.
- _best, _alts (248-268): pick the hypothesis to act on, using the TPRI prior.

Exercise

RUN IT YOURSELF (terminal)

```
python tools/trace.py zoo/rival_1.py reader --seed 3 --deal 0
python tools/trace.py zoo/rival_1.py zoo/liar.py --seed 3 --deal 0
python tools/trace.py zoo/rival_1.py zoo/rival_2.py --seed 3 --deal 0
```

For each: after which call does the hypothesis count reach 1 (or does dead become True)? Is the identified S equal to the true S printed by the trace? Explain why the liar and rival_2 cases differ.

UNDERSTAND IT · answer out loud before the next step

1. Walk through _filter_quote for one hypothesis: when is it kept, when dropped?
2. Why does a liar or a hider make self.dead become True, and what happens then?
3. Why do trust 0.8 and 0.0 get bigger prior weights in TPRI?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for M3 in NOTES.md in your own words.

M4. Valuation and the fallback filter (lines 270-436)

YOU LEARN · **Time:** 75 min · **Goal:** know how the bot values the stock when it is and is not sure

This is the hardest maths in the file. Do it in two passes: first the idea, then the lines.

PROMPT (the idea first)

```
No code changes. Write docs/gauss.md (one page) explaining rival_1's _gauss() for someone
who knows averages but not Kalman filters: (1) the belief about S is a normal distribution
(mean mu, variance v); (2) what one opponent quote does to it, with a number example;
(3) what a buy, a sell and a pass on our quote do (truncated normal), with one example
each; (4) how _state mixes four opponent types by likelihood. Then stop.
```

- _pool (274-285): sums over the unseen cards (count, sum, sum of squares).
- _gauss (287-359): the filter itself. Match each block to a section of docs/gauss.md.
- _state, _state_h (361-405): E = my hand + board + S + hidden board × pm. Exact or fallback.
- _their_F, _my_last, _seat (407-436): the opponent's fair value right now, and who quotes first.

Exercise

RUN IT YOURSELF (terminal)

```
python tools/trace.py zoo/rival_1.py zoo/hider.py --seed 2 --deal 0
```

Against the hider the bot is in fallback mode. Track E against the true value round by round. Where does it get closer, and which event (quote, buy, sell, pass) caused each jump?

UNDERSTAND IT · answer out loud before the next step

1. What is the difference between E (my value of the stock) and F (their fair)? Why does D = E - F drive everything?
2. In _gauss, what does one 'q' event do to mu and v? What does a pass do?
3. Why does the fallback quote around THEIR fair instead of its own?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for M4 in NOTES.md in your own words.

M5. The planner: steer and harvest (lines 438-599)

YOU LEARN · **Time:** 75 min · **Goal:** know exactly how the bot manipulates a reader

- _resp_model (441-472): what their past answers reveal about how they trade (thresholds, 1 or 2 lots).
- _band, _hk, _resp_lots (474-492): how far inside (steer) or outside (harvest) their band to quote.
- _plan (494-598): builds the list of remaining quote events, tries both directions and every steer/harvest mask, fills the 10-lot capacity best edge first, returns the best plan.

Exercise (the key one)

RUN IT YOURSELF (terminal)

```
python tools/trace.py zoo/rival_1.py reader --seed 3 --deal 0
```

1. Mark every quote of ours as steer or harvest (the trace labels them; check two by hand).
2. For one steer: compute how much reader's belief about our hand should move (trust 0.8 × change in our quote's implied hand) and check reader's next quote moved by that much.
3. Explain in two sentences why the plan chose to steer down here, not up.

UNDERSTAND IT · answer out loud before the next step

1. What is a steer quote, and why does it cost nothing?
2. What is a harvest quote, and at what price does the opponent trade?
3. The planner tries every steer/harvest mask. Why at most 2^5 masks, and what fills the 10-lot capacity?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for M5 in NOTES.md in your own words.

M6. The three calls and the swap (lines 600-751)

YOU LEARN · **Time:** 45 min · **Goal:** know what the bot does at each call, end to end

- move, quote, respond, _safe_fair (603-626): the safety nets.
- _move (628-675): two swap rules, one for trust-0 opponents (make the edge as big as possible) and one for readers (follow the position).
- _quote (677-735): first quote, fallback quote (expected-profit search over offsets), exact-mode quote from the plan, and the case where two trust values still disagree.
- _respond (737-751): take their quote only when the plan says so.

Exercise

Without looking, explain to an imaginary organiser what the bot does in each of the three calls, then check yourself against the code. Run UNDERSTAND on anything you got wrong.

UNDERSTAND IT · answer out loud before the next step

1. Why is the first quote E - 0.86 × (my hand - 5 × pm) instead of E?
2. Against a trust-0 opponent, how does the swap rule differ, and why?
3. Every public method is wrapped in try/except. What does each return on an error?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for M6 in NOTES.md in your own words.

5. Wednesday night: a safe upload, and attackers working while you sleep

N1. Upload rival_2, start the red team

TOGETHER · **Time:** 30 min · **Goal:** a strong live entry and overnight test results

1. If you have time, add B1 for rival_2 (the hider detector, section 8) through the experiment loop. If not, upload plain rival_2.

2. Validate and upload. The live board will show how it does against real teams by the morning.
3. Note: the rules require you to explain every line of the FINAL file. You will learn rival_2 tomorrow morning (R1-R6), before the final upload.
4. Paste the attack prompt below. The agents run in the background; read the results in the morning.

W4. Red team and the next-level rival

TOGETHER · **Time:** 60 min · **Goal:** test the bot against opponents built to beat it

PROMPT (attack)

```
Copy moneyheist_bot.py to zoo/champ_v2.py. Then spawn TWO agents in parallel (worktree isolation):
Agent A brief: "Read moneyheist_bot.py, engine.py and rules.py. Your ONLY goal is a bot that
beats moneyheist_bot.py on score. First list its exploitable behaviours ranked by value per
deal, then build and test it (1,000 deals x 3 seeds), save as zoo/red_2.py. It must pass
--validate and stay under 10 ms per call."
Agent B brief: "You are a team in the Hidden Hand competition. Read README.md, rules.py,
engine.py and strategies/*.py, and also zoo/rival_1.py, rival_2.py, rival_3.py: the field
will contain bots like these. Build a bot that beats them AND the house bots. Test with
tools/eval.py. Save as zoo/rival_4.py; under 10 ms per call; must pass --validate."
When both finish: validate, run tools/eval.py moneyheist_bot.py --vs zoo/red_2.py
zoo/rival_4.py --worst 3, and show me red_2's exploit list. Do not change our bot.
```

Then design and code the smallest defence yourself, through the experiment loop. red_2, rival_4 and champ_v2 stay in the pool for good.

UNDERSTAND IT · answer out loud before the next step

1. What did red_2 exploit, and what did you change?
2. Does our bot beat rival_4, which was built knowing rivals 1-3 exist?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for W4 in NOTES.md in your own words.

6. Thursday morning: learn rival_2 as an upgrade

Start with 20 minutes of scouting: our live rank and score, which teams beat us, and one replay of each (section 7 of the Solo Roadmap). Then the modules. Each one starts with a line saying which rival_1 module it builds on: keep zoo/rival_1.py open next to it. The line numbers refer to zoo/rival_2.py.

R1. The opponent model and the hypothesis space (lines 1-107)

YOU LEARN · **Time:** 30 min · **Goal:** know the formula everything else is built on

Builds on M1: coefs() is rival_1's coef() with the trust t left as an unknown, plus a grid of ways an opponent might think.

- Lines 1-39: the docstring, the whole strategy. Read it twice.
- Lines 41-71: constants and priors. For each, say what it controls (they become tuning knobs in W5).
- Lines 74-85: Phi, pdf (the normal distribution) and util (the concave utility).
- Lines 87-107: coefs() and implied(): the opponent's fair value as A0 + A1·t + (B0 + B1·t)·O.

Exercise (pen and paper)

1. Seed 3, round 1: reader holds 3 5 6 Q 4 (O = 30). It saw our quote of 102.64 and quoted 97.9.
2. Rewrite reader.py's fair value with O and t as unknowns, and match it to A0 + A1·t + (B0 + B1·t)·O.
3. Plug in t = 0.8, O = 30 and check you get 97.9 (up to rounding).
4. Ask Claude to run coefs() on that situation and compare with your A0, A1, B0, B1.
5. Compute util(10), util(100), util(-100) with RISK = 60 and explain the shape.

UNDERSTAND IT · answer out loud before the next step

1. Write the opponent's fair value as A0 + A1·t + (B0 + B1·t)·O. What are O and t?
2. What do the five hypothesis dimensions (reads, rule, lam, tau, kappa) each describe?
3. What does util() do to a profit of +10, +100 and -100, and why is that like the square-root score?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for R1 in NOTES.md in your own words.

R2. Rules, the Kalman step, the hypothesis grid (lines 110-199)

YOU LEARN · **Time:** 45 min · **Goal:** know what a hypothesis is and how one update changes it

Builds on M4: ekf() is the same kind of normal-belief update as rival_1's _gauss(), but in 2-D (hand and trust together).

- pick_q (110-126): which of our quotes an opponent of each rule reads, and the clamp.
- ekf (129-141): one Kalman update of the 2-D belief over (O, t).
- mix_into (144-159): blends the update with 'this event was an outlier'.
- __init__, _init (162-193): the hypothesis grid and its priors.
- _prune (195-199): drop hypotheses far behind the best.

Before the line-by-line pass, get the idea of a Kalman filter:

PROMPT (paste into Claude Code)

```
No code changes. Write docs/ekf.md (one page) for someone who knows averages but not Kalman
filters: (1) a belief as a normal distribution (mean, variance), and in 2-D (O and t, with a
covariance); (2) one update from a noisy reading, with a number example in 1-D first;
(3) what the gradient (Ho, Ht) means in ekf(); (4) what mix_into adds (outlier chance).
Then stop.
```

Exercise

Ask Claude how many hypotheses _init creates and why (work it out from the loops first yourself). Then run the trace against reader and watch the count fall as _prune removes the bad ones.

UNDERSTAND IT · answer out loud before the next step

1. What do the four rules RAW, CLAMP, SANE and CUR mean for how an opponent reads our quote?
2. What does one ekf() step do to the mean and spread of (O, t)?
3. How many hypotheses does _init create, and what does _prune throw away?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for R2 in NOTES.md in your own words.

R3. Updating beliefs: _see and _sync (lines 201-362)

YOU LEARN · **Time:** 60 min · **Goal:** know exactly what each event does to every hypothesis

Builds on M2 and M4: _see() is rival_1's _filter_quote() and _gauss() merged: every event updates every hypothesis.

- _state (201-221): the public situation as the opponent saw it in round r.
- _before, _ks (223-245): our position and lots before an event; the sign an ai-style bot uses.
- _see (247-322): the heart of the bot. Quote branch: a noisy reading of their fair. Answer branch: a truncated-normal update from buy / sell / pass, ignoring a pass forced by the position limit.
- _see_swap (324-331): their discard shifts O by (average - card) and widens it.
- _sync (333-362): feeds tape entries and discards in order, and the price being answered right now.

Exercise

RUN IT YOURSELF (terminal)

```
python tools/trace.py zoo/rival_2.py zoo/hider.py --seed 2 --deal 0
python tools/trace.py zoo/rival_2.py zoo/old_plan.py --seed 2 --deal 0
```

Against the hider, watch _mo() stay near the average hand while the true hand is far from it. Explain, using _see's quote branch, why a quote equal to the public estimate is believed. This is the bug B1 fixes.

UNDERSTAND IT · answer out loud before the next step

1. In _see, what happens to every hypothesis when an opponent QUOTE arrives? And when they answer OUR quote?
2. Why do lam = 0 hypotheses get the flat density U_Q for every quote?
3. Why did the hider fool rival_2 before B1, and how does B1 stop it?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for R3 in NOTES.md in your own words.

R4. Valuation and the opponent's belief (lines 364-428)

YOU LEARN · **Time:** 30 min · **Goal:** know how the bot values the stock and predicts the opponent

Builds on M4: _ev() is the same E as rival_1's _state(); _bel() is rival_1's _their_F().

- _ev (364-375): E[V | O] = alpha + beta·O (the hidden board is drawn from what their hand leaves).
- _mo, _p_reader (377-398): weighted mean of O; how sure we are the opponent reads our quotes.
- _slots (401-410): the remaining quote slots and who makes each.
- _bel (412-428): the opponent's fair value under one hypothesis, reading a given quote of ours.

Exercise

In the reader trace, pick one respond call. Compute alpha + beta·_mo() by hand from the printed numbers and check it matches the E the trace prints.

UNDERSTAND IT · answer out loud before the next step

1. What is E[V | O] = alpha + beta·O, and why is beta slightly below 1?
2. What does _p_reader measure, and which decisions use it?
3. In _bel, how does the opponent's fair value depend on the quote of ours that it reads?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for R4 in NOTES.md in your own words.

R5. Play-outs and choosing a quote (lines 430-614)

YOU LEARN · **Time:** 60 min · **Goal:** know how the bot decides what to quote

Builds on M5: instead of enumerating steer/harvest masks, rival_2 simulates the rest of the deal for each candidate price.

- _roll (430-479): a quick simulation of the rest of the deal under one hypothesis, with simple rules for both sides.
- _plan_setup (481-493): the top 6 hypotheses and their weights.
- _quote (523-614): first a risk-neutral search over prices around their predicted fair (imm), then the best few candidates re-scored with roll-outs and util; extreme quotes at ±10; paid probes only when _p_reader is high.

Exercise (the key one)

RUN IT YOURSELF (terminal)

```
python tools/trace.py zoo/rival_2.py reader --seed 3 --deal 0
```

1. Find a quote of 195 or 0 (if any) and the position at that moment. Watch reader's next quote.
2. For one normal quote, ask Claude to print the candidate prices and their scores, and explain why the winner won.
3. Explain in two sentences why the maker loses money in round 1 against a smart taker (it matters for B0).

UNDERSTAND IT · answer out loud before the next step

1. What does _roll simulate, and what simple rules does it assume for both players?
2. In _quote, what is the difference between the first candidate search (imm only) and the second (imm + roll-out + util)?
3. When does rival_2 quote 195 or 0, and why is that free?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for R5 in NOTES.md in your own words.

R6. Respond, swap, safety nets (lines 496-521, 616-686)

YOU LEARN · **Time:** 30 min · **Goal:** know what the bot does at each call, end to end

Builds on M6: same three calls; the swap uses simulated final positions instead of the current position.

- _plain_fair, quote, respond, move (496-521): the safety nets.
- _respond (616-645): buy, pass or sell, each scored with roll-outs and util.
- _move (647-686): estimate the final position with roll-outs, swap the lowest (long) or highest (short) card if the expected gain beats a threshold that falls by round.

Exercise

Without looking, explain to an imaginary organiser what the bot does in each of the three calls, then check yourself against the code. Run UNDERSTAND on anything you got wrong.

UNDERSTAND IT · answer out loud before the next step

1. How does _respond choose between buy 2, pass and sell 2?
2. How does _move estimate the final position, and why do the swap thresholds fall from 15 to 4 by round?
3. Every public method is wrapped in try/except. What does each return on an error?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for R6 in NOTES.md in your own words.

7. Thursday 12:00: gate G, choose the final base

G. Choose the final base

YOU CODE · **Time:** 15 min · **Goal:** one bot to improve and submit

Evidence	Favours rival_2	Favours rival_1
Simulation: weighted pool (eval.py, with B1 on each)	higher weighted score	higher weighted score
Live arena: our rank and the teams we lose to	we lose to bots that hide or steer	we lose to noisy, ai-style or careful teams
Scouting: what the top live bots look like	mostly ai_typical / careful teams	several rival-like bots near the top
Your understanding after M1-M6 and R1-R6	you can explain R3 and R5	R3 and R5 still feel unclear

If it is close, choose the one you can explain better: the rules require it, and you will improve it faster. If you choose rival_1: `cp zoo/rival_1.py moneyheist_bot.py`, put your header back, and use the rival_1 B list below.

UNDERSTAND IT · answer out loud before the next step

1. Which base did you choose, and which two numbers decided it?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for G in NOTES.md in your own words.

8. Thursday afternoon: improve the chosen base

Every change runs through the experiment loop from the Solo Roadmap (section 3): snapshot to moneyheist_prev.py, one change, smoke test, REVIEW, measure with `--compare moneyheist_prev.py`, keep only if the decision rule says so. Do B1 first (already measured, it proves your loop works), then B0.

If the base is rival_2

W3. Improvements, ranked

YOU CODE · **Time:** 3 h · **Goal:** fix the hider blind spot, then stop losing to rival_1 and rival_3

B1. Hider detector · measured: +5.7 vs hider, +0.38 pool, nothing else hurt

Why: a hider quotes the public estimate (board + every other card at the public average). That price is exactly what an honest bot with an average hand would quote, so rival_2 believes 'average hand' every time.

1. Write a function public_at(obs, r): the public estimate when a round-r quote was made (board then, discards up to round r, the other 15 - (cards on board) cards at the public average).
2. In _sync, skip _see for an opponent quote that is within 0.02 of public_at for its round (two places: tape quotes, and the price being answered right now).
3. Expect hider to rise from about +5.3 to about +11; everything else within noise.

B0. Stop losing as the maker against rival_1 and rival_3 · target: both head to heads above 0

What we know: against rival_1, per game, rival_2 loses -42.5 as the maker (round 1 alone -15.4) and wins +21.8 as the taker. Against rival_3: maker -54.6, taker +53.4. Its quotes are being picked off. Already tried and NOT the cause: turning off probes, quote noise, a hand-hiding first quote (see section 1).

1. **Measure first.** Ask Claude for a small script that prints rival_2's PnL by role (maker / taker) and round against any opponent (it exists in this conversation's history). Baseline the numbers above.
2. **Find why its quotes get picked off.** In _quote, the price is chosen from its prediction of how the opponent answers (resp_dist). Compare, quote by quote, what rival_2 predicted rival_1 would do with what rival_1 actually did (ask Claude to log both). Hint: rival_1 does not answer like any of rival_2's hypotheses (it answers from its own plan), so the predicted answer is wrong exactly when it matters.
3. **Fix ideas, one experiment each:** (a) widen the quote (stay further from their predicted fair) when the top hypothesis is weak or the opponent's answers keep surprising us; (b) learn from our own fills: if our quotes keep

getting hit and lose, quote more defensively for the rest of the deal; (c) a hypothesis family for 'smart taker' opponents that trade only with a margin; (d) bigger SIG_R (response noise) as a quick test.

4. Keep only what passes the decision rule on the weighted pool. Watch reader, ai_typical and old_plan: those are rival_2's strengths and must not drop.

B2. Speed check · 2 minutes after every kept change

Run `python tools/speed.py moneyheist_bot.py`. rival_2 starts at 1.4 ms max. Only act if a change pushes it above 10 ms. (The slowest-call line in eval.py is measured under full CPU load and can spike; ignore it.)

B3. Predict the opponent's swap (from rival_3)

A sharp-style opponent that is long throws away a low card (value rises); short, a high card (value falls). rival_3's LO_V / HI_V tables give the expected change by hand sum. Add probability × expected change to E while they have not swapped. Use HINT to find where alpha is built (_ev).

B4. Exact identification for pure readers (from rival_1)

rival_2 already pins reader hands to within 0.5 by round 3, so the gain is small. Only try it if B0 and B3 are done: rival_1's whole-number test in round 1 could sharpen the very first estimate.

B5. Peek (cut first if behind)

rival_2 never peeks. When the top hypotheses say the opponent's quotes are meaningless (lam = 0 weight high) by round 2, a peek (cost 3) might pay. Keep only if the weighted score rises.

If the base is rival_1

B1. Extreme quote at the position limit (from rival_2) · measured +0.26 pool, +0.33 weighted

1. At the top of _quote, after _sync: if position ≥ 10 and round < 5, return 195.0; if position ≤ -10 and round < 5, return 0.0.
2. Expect about +2.1 against ai_typical, +0.9 against liar, +0.4 against hider, roughly 0 elsewhere.

B0. Robust against noisy quoters · target: old_plan from -2.57 to above 0

- Facts (section 1): in round 1 it locks onto a wrong exact hand 14% of the time against old_plan; from round 3 its fallback estimate is 7-9 points off, no better than a blind guess.
- Already failed: 'wait for two quotes before exact mode' (-1.4 pool). Do not retry.
- Fix ideas: learn the quote noise instead of the fixed FB_SQ = 2.5; a hypothesis family for honest-but-noisy quoters (rival_2's lam idea); a model of a level-2 reader (like old_plan's view_of_me); a stricter round-1 test.

Then for either base: B2 speed check (`python tools/speed.py moneyheist_bot.py`) after every kept change, and the defence against what the overnight red team found (N1).

UNDERSTAND IT · answer out loud before the next step

1. Where does rival_2 lose money against rival_1, and what did you change about it?
2. Which improvements did you keep, and what number proves each one?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for W3 in NOTES.md in your own words.

W5. Tune, confirm, upload

CLAUDE BUILDS, YOU REVIEW · **Time:** 45 min · **Goal:** robust constants and a strong entry on the board

PROMPT (paste into Claude Code)

```
List the tunable constants at the top of moneyheist_bot.py. For RISK, SIG_Q, SIG_R, EPS_Q and
P_PROBE (and any constant I added), run a grid of 3-5 values, one at a time, on seeds 1 2,
then re-test the best value of each on seeds 10 11. Table: constant, old, new, gain on 1-2,
gain on 10-11. Do not edit the bot.
```

- Apply only gains that survive on seeds 10-11, yourself.
- Validate, upload, `cp moneyheist_bot.py zoo/champ_v3.py`, tag `upload-v3`.
- Evening scouting: check our live rank, and watch our replay against the top team.

UNDERSTAND IT · answer out loud before the next step

1. Which constants did you change, and did the gain survive on seeds 10-11?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for W5 in NOTES.md in your own words.

9. Thursday evening: lock it in

T1. Last scouting and fixes

TOGETHER · **Time:** 45 min · **Goal:** no surprises from the live board

Morning scouting pass. If a team beats us on the live board, write notes on its behaviour and build zoo/arena_<name>.py with the scouting prompt from the Solo Roadmap. Only fix what the decision rule supports.

T2. Harden: speed and safety

YOU CODE · **Time:** 45 min · **Goal:** a file that cannot crash or time out

1. The try/except safety nets already exist; check each fallback returns something sensible (never the house 105).
2. Confirm nothing at class or module level changes during a deal (public_at from B1 must not store anything).
3. Run `python tools/speed.py moneyheist_bot.py --deals 200`: slowest call under 10 ms.
4. Validate. Compare with champ_v3: no change unless you kept T1 fixes.

UNDERSTAND IT · answer out loud before the next step

1. What is the slowest call now (tools/speed.py), and what protects us if the arena machine is slower?

Stuck on one? Run the UNDERSTAND prompt (Solo Roadmap, section 1). Then write 3 lines for T2 in NOTES.md in your own words.

T3. Final walkthrough: you explain, Claude corrects

YOU CODE · **Time:** 60 min · **Goal:** you can defend every line

PROMPT (paste into Claude Code)

```
Final check. Go through moneyheist_bot.py one function at a time, in file order. For each
function, ask me to explain it first. Only then correct me, pointing to exact lines.
Mark each function green (I explained it fully), amber (partly) or red (could not).
At the end, list the amber and red ones and re-explain them simply.
```

T4. Final upload

YOU CODE · **Time:** 15 min · **Goal:** the entry you want, live, with time to spare

1. Upload at least one hour before the deadline.
2. On the arena, confirm the file is accepted and its first fights show no crash notes.
3. Tag the commit `final`.

Appendix: rival_2 map (line numbers of the unmodified file)

Lines	Section	Module
1-71	Docstring, constants, priors	M1
74-107	Phi, pdf, util, coefs(), implied()	M1
110-199	pick_q, ekf, mix_into, __init__, _init, _prune	M2
201-362	_state, _before, _ks, _see, _see_swap, _sync	M3
364-428	_ev, _mo, _p_reader, _slots, _bel	M4
430-493	_roll, _plan_setup	M5
496-521	Safety nets	M6
523-614	_quote	M5
616-686	_respond, _move	M6

After B1 adds public_at above the class, every later line number shifts by about 10.

Appendix: rival_1 map (zoo/rival_1.py)

Lines	Section	Module
1-93	Docstring, constants, Phi, phi, coef()	M1
95-213	State and history replay	M2
214-269	Exact identification	M3
270-436	Valuation, fallback filter, their fair	M4
438-599	Answer model and the planner	M5
600-751	Safety nets, move, quote, respond	M6